

lab01_report

: 2022056562 서민용, 2023003227 이한주

1. Overall structure

```
module ALU(  
    input [31:0]      operand1,  
    input [31:0]      operand2,  
    input [4:0]        shamt,  
    input [3:0]        funct,  
    output reg [31:0]  alu_result  
);  
    // ... implementations  
endmodule
```

- input으로 들어오는 `funct` 에 따라 `operand1, operand2, shamt` 와 함께 알맞는 ALU operation을 수행하고 ,그 값을 output인 `alu_result` 에 저장하는 간단한 코드입니다.

2. Implementation

```
always @(*) begin  
    alu_result = 0;  
    case (funct)  
        `ALU_ADDU: alu_result = operand1 + operand2;  
        `ALU_AND : alu_result = operand1 & operand2;  
        `ALU_NOR : alu_result = ~(operand1 | operand2);  
        `ALU_OR  : alu_result = operand1 | operand2;  
        `ALU_SLL : alu_result = operand2 << shamt;  
        `ALU_SRA : alu_result = $signed(operand2) >>> shamt;  
        `ALU_SRL : alu_result = operand2 >> shamt;  
        `ALU_SUBU: alu_result = operand1 - operand2;  
        `ALU_XOR : alu_result = operand1 ^ operand2;  
        `ALU_SLT : alu_result = $signed(operand1) < $signed(operand2);  
        `ALU_SLTU: alu_result = (operand1) < (operand2);  
        `ALU_EQ  : alu_result = operand1 == operand2;  
        `ALU_NEQ : alu_result = operand1 != operand2;  
        `ALU_LUI : alu_result = operand2 << 16;  
    endcase  
end
```

- 과제 명세와 2주차 수업내용을 참고하여 verilog 코드를 작성하였습니다.

ALUop	Operations	Action
ALU_ADDU	Add (w/o overflow)	alu_result = operand1 + operand2
ALU_AND	AND	alu_result = operand1 and operand2
ALU_NOR	NOR	alu_result = operand1 nor operand2
ALU_OR	OR	alu_result = operand1 or operand2
ALU_SLL	Shift left logical	alu_result = operand2 << shamt
ALU_SRA	Shift right arithmetic	alu_result = operand2 >> shamt
ALU_SRL	Shift right logical	alu_result = operand2 >> shamt
ALU_SUBU	Sub (w/o overflow)	alu_result = operand1 - operand2
ALU_XOR	XOR	alu_result = operand1 xor operand2
ALU_SLT	Set less than	alu_result = operand1 < operand2
ALU_SLTU	Set less than unsigned	alu_result = unsigned(operand1) < unsigned(operand2)
ALU_EQ	Equal	alu_result = operand1 == operand2
ALU_NEQ	Not equal	alu_result = operand1 != operand2
ALU_LUI	Set upper bits	alu_result = operand2 << 16

Operators in Verilog

- ◆ Arithmetic
 - Binary : +, -, *, /, %
 - Unary : +, -
- ◆ Relational
 - Binary : <, >, <=, >=
- ◆ Equality
 - Binary : ==, !=
 - (comparing include x & z)
 - Binary : ==, !=
 - (returns x if an operand has x or z)
- ◆ Logical
 - Unary : !
 - Binary : &&, ||
- ◆ Bit-wise
 - Unary : ~
 - Binary : &, |, ^, ~^
- ◆ Reduction
 - Unary : &, ~&, ||, ~||, ^, ~^
- ◆ Shift
 - Binary : <<, >>, <<<, >>>
- ◆ Concatenation
 - {a, b, c, ...}
- ◆ Replication
 - {n, {m}}
- ◆ Conditional
 - a?b:c

3. results

- 실행 결과를 더 자세하게 확인하기 위해 `ALU_tb.v` 에 다음과 같이 코드를 추가하여 모든 signal들에 대해서 `waveform.vcd` 파일을 만들어주었습니다.

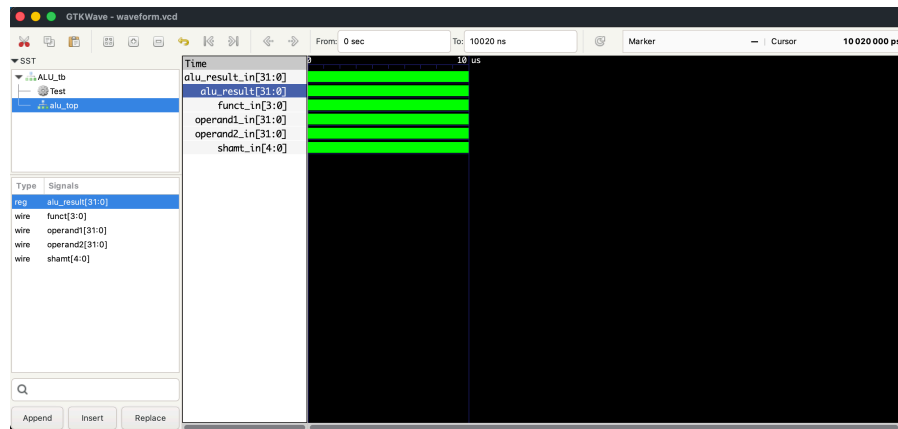
```
initial begin
    $dumpfile("waveform.vcd");
    $dumpvars(0,ALU_tb);
end
```

- 또한 mem 파일에 담긴 test case의 개수가 총 10000개임을 확인하여 모든 경우를 확인하기 위해 `ALU_tb.v` 의 `TEST_SIZE` 를 10000으로 설정하고 실행하였습니다.

```
minyongseo@Minui-MacBookAir HW01 % iverilog ALU.v ALU_tb.v
minyongseo@Minui-MacBookAir HW01 % ./a.out
VCD info: dumpfile waveform.vcd opened for output.
PASSED =      10000, FAILED =          0
ALU_tb.v:80: $finish called at 10020000 (1ps)
minyongseo@Minui-MacBookAir HW01 %
```

터미널에 출력되는 텍스트 양을 줄이기 위해 `$display` 코드를 일부 주석처리 하였습니다.

- 실행결과 `PASSED` 가 10000으로 모든 경우에 대해 통과함을 확인했습니다.
- GTKWave를 통해서 실행결과의 자세한 내용을 확인 할 수 있었습니다.
- 우선 실행시간에 대해서, 분석해보자면, 총 10020ns가 걸렸으며, 10ns부터 변수들의 값이 바뀌기 시작하고, 그 이후 1ns 마다 값이 바뀌며 10010ns 까지 각 테스트에 대해서 연산합니다.



- 이는 코드와 비교했을 때 매우 정상적으로 나왔음을 확인할 수 있는 부분이었습니다.
 - Skeleton Code의 `ALU_tb.v` 기준 의 일부 코드입니다.

```
task Test;
    input [31:0] operand1_in;
    input [31:0] operand2_in;
    input [4:0] shamt_in;
    input [3:0] funct_in;
    input [31:0] alu_result_in;
    begin
        operand1 = operand1_in;
        operand2 = operand2_in;
        shamt = shamt_in;
        funct = funct_in;
        $display("TEST CTRL Sig: %h :", funct);
        #1;
        // ...
    end
endtask
// ...
initial begin
    i = 0;
    PASSED = 0;
    FAILED = 0;
    #10

    for (i = 0; i < TEST_SIZE; i = i + 1) begin
        Test(operand1_mem[i], operand2_mem[i], shamt_mem[i],
        funct_mem[i], alu_result_mem[i]);
    end

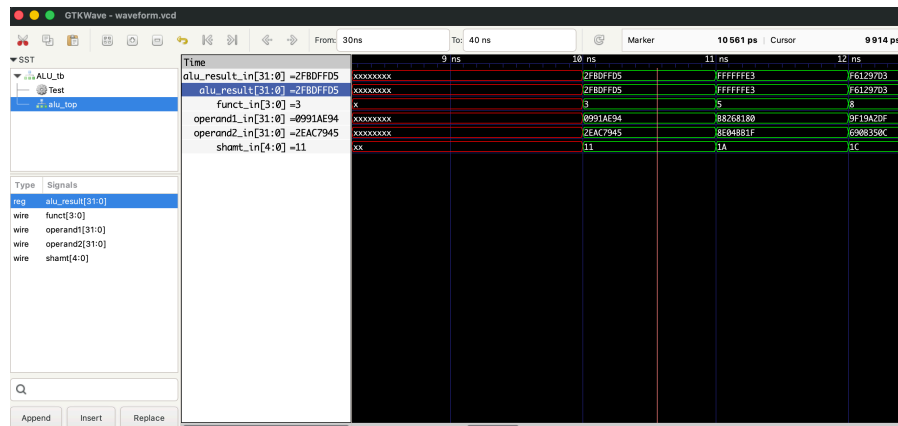
    #10
    $display("PASSED = %d, FAILED = %d", PASSED, FAILED);
```

```

$finish;
end

```

- timescale이 1ns로 명시 되어있어 각 Test 마다 1ns가 걸리고, 모든 Test를 시작하기전 10ns를 대기하고 모든 Test가 끝나면 10ns를 대기하도록 작성되어 있음을 확인 할 수 있습니다.
- 첫 test case만 확인해보자면,



- `funcnt` 의 값이 3으로, ALU_OR 연산입니다.
- op1이 `0991AE94` , op2가 `2EAC7945` 로 각각 binary로는 `0000 1001 1001 0001 1010 1110 1001 0100` , `0010 1110 1010 1100 0111 1001 0100 0101` 로 OR 연산 시 `0010 1111 1011 1101 1111 1111 1101 0101` 로 hexadecimal로는 `2FB0FFD5` 가 맞습니다.
- 연산 결과인 `alu_result[31:0]` 의 값과 정답인 `alu_result[31:0]` 이 모두 `2FB0FFD5` 로 일치하여 `PASSED` 처리 되었음을 확인 할 수 있습니다.

4. Troubleshooting

- 초기엔 변수의 부호를 고려하지 않아 `ALU_SRA` 와 `ALU_SRA` 에 대해서는 테스트 실패를 받았습니다.
- 이는 verilog 변수의 기본형은 `unsigned` 이기 때문에, MSB가 뒤로 밀리게 되는 `ALU_SRA` 와 대소비교를 하는 `ALU_SLT` 에 대해서는 `$signed()` 를 취해 부호 포함 연산임을 명시해주어야 했던 문제였습니다.